

OCP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 14

Creating and Using interfaces

CHAPTER OVERVIEW

An interface is used to describe behavior as captured by a group of methods. It doesn't tell you anything about the object behind the behavior except that the methods declared in the interface can be invoked on that o...

EXAM OBJECTIVES COVERED

- ◆ Create and use interfaces
- ◆ Identify functional interfaces
- ◆ Utilize private, static, and default interface methods

Chapter 14 Creating and Using interfaces

Exam Objectives

- Create and use interfaces
- Identify functional interfaces
- Utilize private, static, and default interface methods

14.1 Create and implement interfaces

14.1.1 Using interfaces

An **interface** is used to describe behavior as captured by a group of methods. It doesn't tell you anything about the object behind the behavior except that the methods declared in the interface can be invoked on that object. This group of methods, therefore, is a way to interact with the object.

An interface is defined using the keyword "**interface**". For example:

```
interface Movable{  
    void move(int x);  
}
```

From a purely OOP perspective, an interface should not contain any implementation. It should only contain method declarations. However, Java has always permitted interfaces to contain static fields. Furthermore, from Java **8**, Java has permitted an interface to contain **default methods** and **static methods** and Java **9** has allowed an interface to contain **private methods** (static as well as non-static) as well.

There are several rules about interfaces that you must learn by heart. So, let me go over them one by one.

Everything is public in an interface by default 🙌

Everything declared inside an interface is implicitly **public** except methods that are explicitly declared **private**. This means that members of an interface are always public irrespective of whether you define them as public or not. In fact, you are prohibited from defining them as private or protected. The following example illustrates this point:

```
interface Movable{  
    void move1(int x); //OK, move1 is implicitly public  
    public void move2(int x); //OK, move2 is explicitly public
```

```
    private void move3(int x); //NOT OK, abstract methods are always  
    public  
    protected void move4(int x); //NOT OK, abstract methods are  
    always public  
    private int PVT_VALUE = 10; //NOT OK, fields cannot be protected  
    or private
```

```
    int VALUE = 10; //OK, VALUE is implicitly public  
    private void pvtMethod(){ }; //OK, non-abstract methods can be  
    private (but not protected)  
}
```

This makes sense because the whole purpose of an interface is to put out a way to interact with an object in front of the world. Remember that an interface is not about how an object is implemented. It is about how an object can be interacted with. In that sense, an interface is a **contract** between an object and the rest of the application components and since everything in a contract must be public, everything in an interface must be public.

What if you want to disclose members of an interface only to the members of the same package? Well, make the interface "**default**". As discussed earlier, default access allows something to be visible only to members of the same package. If the interface itself is not visible outside the package, its members certainly won't be.

Private methods do not really fit conceptually in an interface but the need for private methods was felt after default methods were introduced in Java 8. If a method gets too big or if there multiple methods with a lot of common code, there was no way to refactor them into smaller methods without exposing all of them to the world because everything in an interface had to be public. Java 9 fixes this problem by allowing **private methods** in an interface.

An interface is always abstract [👉](#)

As mentioned above, an interface is merely a contractual description of some behavior. It is not an entity that can actually fulfill that contract. For example, you may describe the behavior of movement using an interface named Movable but you need an object such as an animal or vehicle to exhibit that behavior. Therefore, it does not make sense to instantiate an interface. For this reason, an interface is **implicitly abstract**. Although legally valid, it would be redundant to declare an interface as abstract.

Variable definitions in an interface [👉](#)

All variables defined in an interface are implicitly **public**, **static**, and **final**. The following example illustrates this point:

```
interface Movable{
    int UNIT1 = 1;
    static int UNIT2 = 1;
    static final int UNIT3 = 1;
    public static final int UNIT4 = 1;
```

```
}
```

All four of the variables declarations above are valid. All of the variables are **public**, **static**, and **final** even though the first three have not been declared as such. This implies that you cannot have instance variables in an interface.

Remember that instance variables are meant to store **state**, which means they are really a part of **implementation**. Therefore, instance variables have no place in an interface. To be honest, it is not a good idea to have static variables either in an interface but for reasons best known to the language designers, Java allows static constants in an interface. Many people use this feature to define "**global constants**" but it is simply a bad design choice. Avoid them in your code.

Methods in an interface

An interface can have four kinds of methods :

1. **abstract methods** - You have seen abstract methods in the previous section about abstract classes. They contain just the declaration and no body. It is the same thing here except that the keyword **abstract** is optional. For example:

```
interface Movable{  
    void move1(int x); //implicitly abstract  
    abstract void move2(int x); //explicitly abstract  
}
```

Both of the methods declared above are abstract and are implicitly public.

2. **default methods** - Default methods are a way for an interface to include a default implementation for a method. They are defined using the keyword **default** as follows:

```
2. interface Movable{
    default void move(int x){
        System.out.println("Dummy implementation. Moving by "+x+"
points");
    }
}
```

Observe that default methods are just the opposite of abstract methods - abstract methods cannot have an implementation while the whole purpose of default methods is to provide an implementation. Therefore, a method cannot be default as well as abstract at the same time. Thus, the following code will not compile:

```
interface Movable{
```

```
//must be marked default or private because it has a body
void move1(int x){
    System.out.println("Dummy implementation. Moving by "+x+"
points");
}
```

```
void move2(int x); //ok, no body

}
```

Default methods are always public even if you do not declare them as such explicitly. You cannot mark them `private` or `protected`.

3. **static methods** - As the name implies, static methods belong to the interface itself and not to the object implementing that interface. They are defined using the keyword **static** as follows:

```
interface Movable{
```

3.

```
static void sayHello(){
    System.out.println("Hello!");
}
}
```

Since `sayHello` is a static method, you don't need an instance of any object to invoke it. You can invoke it directly on the interface just like you invoke static methods on a class. For example:

```
public class TestClass{
    public static void main(String[] args){
        Movable.sayHello();
    }
}
```

Static methods can be marked `public` or `private` but not `protected`. If no access modifier is specified, they are implicitly `public`. Static methods cannot be marked `default` either.

4. **private methods** - Private methods (static as well as no-static) have been allowed since Java 9 and, as explained above, they are helpful when a default static methods gets too big and needs to be refactored into smaller internal methods without exposing the internal methods to the outside world. Here is an example:

```
interface Movable{
    private void moveInternal(){ //don't want to make it
accessible to others
        System.out.println("in moveInternal");
    }
    default void move(int n){
        while(n-- > 0) moveInternal();
    }
}
```

4. Note that since private methods are not visible outside the interface, they cannot be marked `default`.

Since methods of an interface are meant to be implemented by the classes that implement the interface, methods of an interface cannot be declared `final`. Observe that this is opposite to the rule that is applied to variables. Variables of an interface are always `final`.

Although not important for the exam, it is possible to define classes, records, interfaces, and enums within an interface. They cannot be anything other than public.

Marker interface 🙌

You may encounter an interface that does not contain anything at all. Such interfaces are called "marker interfaces". For example:

```
interface SpecialThing{  
}
```

In the above code, `SpecialThing` is a marker interface.

The purpose of a marker interface is to tag a class with an extra piece of information about that class itself. For example, if any class implements `SpecialThing`, it implies that that class is a `SpecialThing`. This information, also called "metadata", could be used by some code or some tool that treats all `SpecialThing`s in a certain way.

A well known marker interface in Java is `java.io.Serializable`. It signifies to the JVM that objects of classes implementing this interface can be serialized and deserialized.

14.1.2 Implementing an interface 🙌

A class can implement any number of interfaces by specifying their names in its **implements** clause. For example, in the following code the `Price` class implements two interfaces:

```
interface Movable{
    void move();
}
interface Readable{
    void read();
}
class Price implements Movable, Readable{
    public void move() { System.out.println("Moving ... "); }
    public void read() { System.out.println("Reading ... "); }
}
```

Of course, once a class declares that it implements an interface, it must then have the implementation for all of the abstract methods declared in that interface. It could either implement the methods itself or inherit them from its ancestor class. The signature of the implemented methods must be compatible with the signatures of the abstract methods of the interface. This means:

1. The method must be explicitly defined as **public** because abstract methods of an interface are implicitly public and you cannot reduce accessibility of a method.
2. The return type must be **covariant** with the return type declared by the interface method just like when overriding a method.
3. Parameter types must be **exactly** the same as in the method in the interface. Remember that return types are covariant, parameter types are not.
4. The **throws** clause must be compatible with the throws clause of the interface method. The same rules that you saw while overriding a method in a subclass earlier are applicable here as well, which means, the implementing method is not allowed to throw a wider checked exception than the one specified by the interface method.

5. Although not mandatory, it is a good practice to apply **@Override** annotation to the method in the class.

If an interface provides a default implementation for a method in the form of a default method, the implementing class does not necessarily have to provide implementation for that method.

If the class does not have implementation for even one of the abstract methods declared in the interface that it says it implements, the class must be declared **abstract**. Otherwise, the compiler will refuse to compile the class.

So, for example, if the `StockPrice` class does not provide an implementation for the `read` method declared in `Readable`, it would have to be declared abstract like so:

```
abstract class StockPrice implements Movable, Readable{  
    public void move() { System.out.println("Moving ... "); }  
}
```

A class is allowed to extend another class as well as implement any number of interfaces at the same time. In this case, the class declaration will have an **extends** clause as well as an **implements** clause. For example:

```
interface Printable{  
    void print();  
}  
public class StockPrice extends Price implements Printable{  
    @Override  
    public void print(){  
        System.out.println("Printing StockPrice ... ");  
    }  
}
```

In the above code, `StockPrice` implements `Printable` explicitly and since it extends `Price`, it implements `Movable` and `Readable` implicitly. Furthermore,

`StockPrice` inherits the move and read methods from `Price` and therefore, it does not need to define them again.

Note that the order of extends clause and implements clause is important. The extends clause must appear before the implements clause.

Inheritance of static methods of an interface 🙌

Unlike the static methods of a class, the static methods of an interface cannot be inherited. This difference is illustrated by the following code:

```
class Price{
    static void m(){
        System.out.println("In Price.m()");
    }
}
interface Printable{
    static void p(){
        System.out.println("In Printable.p()");
    }
}
class StockPrice extends Price implements Printable{
    void someMethod(){
```

```
        p(); //will not compile
```

```
    }
}
class TestClass{
    public static void main(String[] args){
        StockPrice.m(); //works fine
        Printable.p(); //works fine too
```

```
StockPrice.p(); //will not compile
```

```
}  
}
```

In the above code, since `StockPrice` extends `Price`, `StockPrice` inherits the static method `m()` defined in `Price`. However, even though `StockPrice` implements `Printable`, it does not inherit the static method `p()` defined in `Printable`. That is why the invocations `p();` in `StockPrice` and `StockPrice.p()` in `TestClass` fail to compile. To invoke a static method of an interface, one must use the `<interface name>. <method name>` syntax, i.e., `Printable.p();` even within the class that implements the interface.

Inheritance of multiple versions of a default method 🙌

Since it is possible for a class to implement multiple interfaces, it is possible for a class to inherit multiple implementations of a default method from more than one interface. For example:

```
interface Task{  
    public default void doIt(){  
        System.out.println("Doing Task");  
    }  
}  
  
interface Activity{  
    public default void doIt(){  
        System.out.println("Doing Activity");  
    }  
}
```

```
//will not compile  
class Process implements Task, Activity{}
```

In the above code, `Process` implements two interfaces and since each of them contains a default method named `doIt` with the same signature, `Process` now has two implementations for the same method. This is a problem because when you call `doIt` method on a `Process` object, the JVM will not be able to determine which implementation of `doIt` to invoke.

Java resolves this problem by forcing the class to provide an implementation of the method of its own to remove the ambiguity in invocation.

```
class Process implements Task, Activity{
    public void doIt(){
        System.out.println("Doing Process");
    }
}
```

The JVM now has only one implementation of `doIt` method to invoke and so, the above code compiles fine.

Let us look at a case that is even more interesting. Can you tell what happens when you try to compile the following code:

```
interface Task{
    int SIZE = 10;
    default void doIt(){
        System.out.println("Doing Task");
    }
}

interface Activity{
    long SIZE = 20;
    void doIt();
}

class Process implements Task, Activity{
```

```

public static void main(String[] args){
    Process p = new Process();
    p.doIt();
}
}

```

Observe that `doIt` method is declared by both the interfaces but only one of them provides an implementation. Thus, `Process` inherits only one implementation for `doIt` method. There is no ambiguity for the JVM in determining which implementation of `doIt` to invoke. Therefore, it should compile, right? Wrong!

Java designers thought that even though there is only one implementation for the `doIt` method, there is no guarantee that this implementation is appropriate for the `doIt` method declared by another interface. The contract of the other interface could possibly be different from the one that provides the implementation for the `doIt` method. Therefore, it is better for the class to explicitly provide an implementation of the `doIt` method.

Note that this issue does not arise when there is no default implementation available at all because in that case the class will have to provide its own implementation anyway.

Inheriting a default method and a method from a super class with the same signature 🙌

When a class inherits a method from its superclass and also from an interface that it implements, the version that it inherits from a superclass overrides the default method defined in any of the interfaces that it implements. For example, the following code prints Running process instead of Doing task:

```

interface Task{
    default void doIt(){ System.out.println("Doing task"); }
}
class BaseProcess{
    void doIt(){ System.out.println("Running process"); }
}

```

```

}
class Process extends BaseProcess implements Task{
    public static void main(String[] args){
        Process p = new Process();
        p.doIt();
    }
}

```

Inheritance of multiple versions of a variable 🙄

The situation with the `SIZE` variables in the above code is a bit special. Fields of an interface are inherited by a sub class and therefore `Process` does get two versions of `SIZE` variable. Java allows a class to inherit multiple fields with the same name as long as you don't try to use those fields ambiguously. This means that you will get a compilation error only if you try to use `SIZE` within `Process` directly without specifying which `SIZE` field are you trying to refer to. This is illustrated below:

```

interface Activity{
    long SIZE = 20;
    //void doIt(); //let's get rid of it to avoid the method
    //ambiguity issue for now
}
class Process implements Task, Activity{
    public static void main(String[] args){
        System.out.println(SIZE); //will not compile
    }
}

```

The above code will fail to compile with an error message, "reference to `SIZE` is ambiguous" because the compiler is not able to figure out which `SIZE` you are trying to use. But the following code will compile fine because the compiler has no confusion about the `SIZE` that are you referring to here:

```

class Process implements Task, Activity{
    public static void main(String[] args){
        System.out.println(Activity.SIZE);
        System.out.println(Task.SIZE);
    }
}

```

14.1.3 Extending an interface 🙋

It is possible for an interface to **extend** any number of interfaces. For example:

```

interface Readable{
    int SIZE = 0;
    void read();
}
interface Writable{
    void write();
}
interface ReadWritable extends Readable, Writable{
    //inherits SIZE and read() from Readable
    //inherits write() from Writable
    void delete();
}

```

Remember that a class cannot extend an interface, it can only implement an interface. Whereas, an interface cannot implement any interface it can only extend an interface. Thus, the following definitions of Writer will not compile:

```

interface Writer implements Writable{ //interface cannot
    "implement" any interface
    public void write(){ }
}

```



```

}
class Writer extends Writable{ //class cannot "extend" any
    interface
    public void write(){ }
}

```

The extending interface inherits all the members **except static methods** of each of the other extended interfaces.

Inheriting multiple versions of a default method 🙌

It is possible for an interface to inherit a field or an abstract method with the same signature from two of its super interfaces. But inheriting multiple default methods or one default and one or more abstract methods with the same signature has the same problem that you saw earlier in a class that implements multiple interfaces. Java does not allow it. Here is an example:

```

interface Readable{
    int SIZE = 10;
    void read();
    static void staticMethod(){
        System.out.println("In Readable.staticMethod");
    };
    default void defaultMethod(){
        System.out.println("In Readable.defaultMethod");
    };
}

interface Writable{
    int SIZE = 20;
    void write();
    static void staticMethod(){
        System.out.println("In Writable.staticMethod");
    };
}

```

```

};

/* commenting the following two methods out
default void defaultMethod(){
    System.out.println("In Writable.defaultMethod");
};
void defaultMethod();
*/
}
interface ReadWritable extends Readable, Writable{
    //inherits SIZE, read(), and defaultMethod() from Readable
    //inherits SIZE and write() from Writable
}

```

The above code compiles fine. But if you uncomment either of the `defaultMethod`s in `Writable`, `ReadWritable` will fail to compile because it would be inheriting two different implementations (or one implementation and one declaration) of `defaultMethod`. The solution is the same as what we did with the class. `ReadWritable` must provide its own implementation of the `defaultMethod` to resolve the ambiguity:

```

interface ReadWritable extends Readable, Writable{
    default void defaultMethod(){
        System.out.println("In ReadWritable.defaultMethod");
    };
}

```

Observe that `staticMethod` is also defined in both the super interfaces but it does not cause any problem because static methods of an interface are never inherited, which means `ReadWritable` does not get even a single implementation of `staticMethod` from either of its superinterfaces, let alone two, which would cause ambiguity.

Inheriting multiple variables with the same name 🙅

The situation with the `SIZE` variable (remember that it is implicitly static and final) is, again, the same as what you saw in a class that inherits multiple versions of a field from multiple interfaces. Static fields of an interface are inherited by a sub interface and therefore `Readable` does get two versions of `SIZE` variable. Java allows an interface to inherit multiple fields with the same name as long as you don't try to use those fields ambiguously. For example, the following code will not compile :

```
interface ReadWritable extends Readable, Writable{
    int NEWSIZE = SIZE; //will not compile because SIZE is being used ambiguously
}
```

The above code will fail to compile with an error message, "reference to `SIZE` is ambiguous" because the compiler is not able to figure out which `SIZE` are you trying to use. But the following code will compile fine because the compiler has no confusion about the `SIZE` that you are referring to here:

```
interface ReadWritable extends Readable, Writable{
    int NEWSIZE = Readable.SIZE; //fine
}
```

14.1.4 Functional interfaces 🙅

Any interface that has exactly one abstract method is a functional interface. It may have other private, static, or default methods but it must have one and only one abstract method. This abstract method is also called the "functional method" of that functional interface.

The following are examples of valid functional interfaces:

```
@FunctionalInterface
interface Mover1{
    void move(int x, int y);
}

@FunctionalInterface
interface Mover2{
    void move(int x, int y);
    default void no_op(){ System.out.println("do nothing"); }
}
```

And the following are examples of invalid functional interfaces:

```
@FunctionalInterface //valid interface but not a valid functional interface
interface Mover3{ //because it has two abstract methods
    void move(int x, int y);
    void move3D(int x, int y, int z);
}

@FunctionalInterface //valid interface but not a valid functional interface
interface Mover4{ //because it has no abstract method
    default void move(int x, int y){
        System.out.println("moving "+x+" "+y);
    }
}

@FunctionalInterface //valid interface but not a valid functional interface
//NewMover has two abstract methods including the one that it inherits from Mover2
interface NewMover extends Mover2{
    void m2(String str);
}
```

Observe that I have used `@FunctionalInterface` annotation in the examples above. It lets the compiler verify that an interface satisfies all the requirements of a function interfaces and generate an error if it does not. Without this annotation, all of the interfaces defined above will compile without any error because they are valid interfaces.

You will see more of functional interfaces soon.

14.1.5 Instantiating abstract classes and interfaces 🙌

Abstract classes and interfaces are abstract. Objects of their kind do not exist, which is why they are called abstract in the first place. Therefore, they cannot be instantiated, period.

I have seen many students getting confused when they see code that looks like it is instantiating an abstract class. For example:

```
abstract class Animal{
    public static void main(String[] args){
        Animal a = new Animal(){ };
    }
}
```

The above code seems to be instantiating `Animal` even though `Animal` is abstract! The following is an example that seems to instantiate an interface:

```
interface Dummy{
}
public class TestClass{
    public static void main(String[] args){
        Dummy d = new Dummy(){ };
    }
}
```

```
}  
}
```

Well, first thing that you should observe is the presence of `{ }` between `new Animal()` and `;`. This is not the syntax for instantiating a class. For instantiating `Animal`, you would have to write `new Animal();` and if you try to do that you will get a compilation error that says, "`Animal is abstract; cannot be instantiated`".

In fact, the above code uses the syntax for declaring as well as instantiating a concrete inner class that extends `Animal` without giving this class a name. Since no name is given to this class, it is called an **anonymous class**. When the compiler sees this code, it actually creates a class, gives this class a weird looking name, and generates a separate class file for this class. It is this class that is instantiated at run time. Since `Animal` class doesn't have any abstract method, this anonymous class doesn't need to implement any method. Had `Animal` had an abstract method, you would have to implement this method inside the curly braces.

The same thing is happening in the case of the interface example above. The code is defining and instantiating, in a single statement, an anonymous class that implements the `Dummy` interface.

You are not expected to know all the nitty-gritties of anonymous classes for the exam. Just remember that anonymous classes are never abstract and there is no way to extend them. They cannot have an explicitly defined constructor either.

14.2 Distinguish class inheritance from interface inheritance including abstract classes

14.2.1 Difference between Interface and Abstract Class 🙌

"What is the difference between an **interface** and an **abstract class**?" is usually the first question that is asked in a Java "tech-check". While being a simple ice breaker, it also an excellent question to judge a candidate's understanding of OOP.

Candidates usually start parroting the technical differences such as an interface cannot have method implementations (which it can, since Java 8), while abstract class can. An interface cannot have static methods (which it can, since Java 8) or instance fields while an abstract class can and so on.

All that is correct, but is not really impressive. The fundamental difference between an interface and an abstract class is that an interface defines just the behavior. An interface tells you nothing about the actual object other than how it behaves. An abstract class, on the other hand, defines an object, which in turn, drives the behavior.

If you understand this concept everything else about them will fall in place. For example, "movement" is a behavior that is displayed by various kinds of objects such as a Car, a Cat, or a StockPrice. These objects have no commonality except that they move. Saying it the other way round, if you get an object that "moves", you don't get any idea about what kind of an object are you going to deal with. It could be a Car, a Cat, or a StockPrice. If you were to capture this behavior in Java, you would use an interface named **Movable** with a method named **move()**.

On the other hand, if you talk about Automobile, a picture of an object starts forming in your head immediately. You can sense that an Automobile would be something that would have an engine, would have wheels, and would move. You intuitively know that a StockPrice or a Cat cannot be an Automobile even though they both do move. An abstract class is meant exactly for this purpose, when, once you identify a conceptual object, you do not need to worry about its behavior. The behavior kind of flows automatically. If you create an abstract class named Automobile, you are almost certain that it would have methods such a move, turn, accelerate, or brake. It would have fields for capturing inner details such Engine, Wheels, and Gears. You get all

that just by saying the word Automobile.

From the above discussion, it should be clear that interfaces and abstract classes are not interchangeable. Even though an abstract class with no non-abstract method looks functionally similar to an interface, both are fundamentally different. If you are capturing a behavior, you must use an interface. If you are capturing a conceptual object, you must use an abstract class.

14.3 Exercise

1. Given the following two classes:

```
public class TestClass{
    public static void main(String[] args){
        Document d = new PdfDocument();
        System.out.println(d.getType()); //should print "pdf"
    }
}
class Document{
    private String type = "dummy";
    private byte[] data;
    //insert appropriate getters and setters
}
```

The above code refers to a class named PdfDocument . Write code for this class such that TestClass will print "pdf" when executed.

2. Given the following code:

```
class Radio {
    private double frequency=1.1;
    //insert appropriate getter and setter
}
class TV {
```


2.

```
private int channel = 5;
//insert appropriate getter and setter
}
public class TestClass{
    public static void main(String[] args){
        TV t = new TV();
        Radio r = new Radio();
        reset(t);
        reset(r);
        System.out.println(r.getFrequency()); //should print 0.0
        System.out.println(t.getChannel()); //should print 0
    }
}
```

Write code for the reset method in `TestClass` such that `TestClass` will print `0.0` and `0` when executed.

3. You are expected to reset several electronic devices in future. Refactor the code given above such that `TestClass`'s `reset` method is able to reset any new device without requiring any change in the method code.
4. Given the following code:

```
class Pie{
    public void makePie(){
        System.out.println("making pie");
    }
}
```

Create two classes `PumpkinPie` and `ApplePie` that extend `Pie`. Override the `makePie` method in these classes. Ensure that the overridden method is also invoked whenever `makePie` is invoked on objects of these classes.

5. Add a static method named `getCalories` in `Pie`:

```
public static int getCalories(){
    return 100;
}
```

5. Create a new class named `Nutritionist` with a method named `printCalories`. This method should take any kind of `Pie` and print the correct number of calories as applicable for the given kind of `Pie`.
6. Given the following two classes:

```
class XMLTransformer {  
    public String transform(String Data){ return "xmldata"; }  
}  
  
class NetworkTransformer {  
    public String transform(String Data) throws IOException { return  
        "data from network"; }  
}
```

Refactor the two classes by adding an abstract super type named `Transformer`.

7. Create a class named `TransformerFactory` with a method named `getTransformer`. This method should return different types of transformers based on an argument. Use this factory class to get different types of transformers and invoke the transform method on them.
8. Create a method that takes an array of strings and returns an `ArrayList` containing the same strings.
9. Update the above method to remove the duplicate elements from the `ArrayList` before returning.
10. Create a method with the signature `switchIt(ArrayList al, int a, int b)`. This method should return the same list but after switching the elements at positions `a` and `b`.